Day 3 — Object-Oriented Programming (OOP)

Topics

 Classes and objects

 Constructors

 Encapsulation

 Inheritance

 Polymorphism

 Abstraction

 Access modifiers

 Methods and method overloading

Tasks

 Create Student and Employee classes.

 Add properties and methods.

 Demonstrate constructors, inheritance, and encapsulation.

Expected Output

 Java OOP practice programs.

 Console-based Student Management Application.

1. Classes and Objects

Class kya hoti hai?

Definition: A class is a blueprint or template used to create objects. It defines the data (fields) and behaviour (methods) of objects.

Hinglish mein: Class ek design ya blueprint hoti hai jiske according hum objects banate hain.

Real-life example: Ghar ka blueprint ek class ki tarah hai. Us blueprint se banne wale alag-alag ghar objects ki tarah hain.

- Class = Car ka design
- Object = Actual BMW, Audi ya Tata car
- Properties = color, brand, speed
- Behaviours = start(), stop(), accelerate()

Java example

```
class Student {
    String name;
    int age;

    void study() {
        System.out.println(name + " is studying");
    }
}

public class Main {
    public static void main(String[] args) {

        Student s1 = new Student();

        s1.name = "Suhana";
        s1.age = 21;

        System.out.println(s1.name);
        System.out.println(s1.age);

        s1.study();
    }
}
```

Output:

```
Suhana
21
Suhana is studying
```

Code ko samjho

1. `class Student` — Student naam ki class banayi.
2. `String name` aur `int age` — object ki properties define ki.
3. `study()` — object ka behaviour define kiya.
4. `new Student()` — ek naya Student object banaya.
5. `s1.name` — object ki property access ki.
6. `s1.study()` — object ka method call kiya.

`Student s1 = new Student();` mein `s1` reference variable hai jo object ko refer karta hai, aur `new Student()` object create karta hai.

Ek hi class se multiple objects bana sakte hain:

```
Student s1 = new Student();
Student s2 = new Student();

s1.name = "Suhana";
s2.name = "Priya";
```

Dono objects ki `name` values alag hongi.

Interview answer: A class is a blueprint that defines the properties and behaviours of objects. An object is an instance of a class with its own state.

2. Constructors

Constructor kya hota hai?

Definition: A constructor is a special member of a class used to initialize an object when it is created.

Hinglish mein: Constructor ka use object banate waqt uski initial values set karne ke liye hota hai.

Constructor ke important rules

- Constructor ka naam class ke naam jaisa hi hona chahiye.
- Constructor ka koi return type nahi hota, `void` bhi nahi.
- Object create hone par constructor invoke hota hai.
- Constructors overloaded ho sakte hain.
- Constructors inherit nahi hote.

Example

```
class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 18;
    }

    void display() {
        System.out.println(name + " " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.display();
    }
}
```

Output:

```
Unknown 18
```

Jab `new Student()` execute hua, tab `Student()` constructor ne initial values set kar di.

Types of constructors

A. Default constructor

Agar aap class mein koi constructor declare nahi karte, Java compiler automatically ek default no-argument constructor provide karta hai. Yeh fields ko unki default values par chhodta hai, jaise `int` ke liye `0` aur reference fields ke liye `null`.

B. No-argument constructor

Programmer khud zero-parameter constructor likhta hai.

```
Student() {
    name = "Suhana";
}
```

C. Parameterized constructor

Constructor parameters ke through values receive karta hai.

```
class Student {
    String name;
    int age;

    Student(String n, int a) {
        name = n;
        age = a;
    }

    void display() {
        System.out.println(name + " " + age);
    }
}
```

Object creation:

```
Student s1 = new Student("Suhana", 21);
Student s2 = new Student("Priya", 22);

s1.display();
s2.display();
```

Output:

```
Suhana 21
Priya 22
```

Yahan `n` ko `name` mein aur `a` ko `age` mein assign kiya gaya.

Constructor overloading

Ek class mein multiple constructors ho sakte hain, bas unke parameter lists different hone chahiye.

```
class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 0;
    }

    Student(String name) {
        this.name = name;
        age = 0;
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

`this.name` current object ke field ko refer karta hai. Yeh tab useful hai jab parameter aur field ka naam same ho.

Interview answer: A constructor initializes an object when it is created. It has the same name as the class and has no return type.

Important difference: Constructor object initialize karta hai; method koi specific task perform karta hai. Constructor ko normal method ki tarah call nahi karte.

3. Encapsulation

Encapsulation kya hai?

Definition: Encapsulation is the process of combining data and methods into one unit and controlling access to the internal data.

Hinglish mein: Data ko class ke andar protect karna aur usko controlled methods ke through access ya update karna encapsulation hai.

Real-life example: Bank account mein aap directly balance ko change nahi karte. Aap deposit ya withdraw karte ho, jahan rules check kiye ja sakte hain.

Java example

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        account.deposit(5000);

        System.out.println(account.getBalance());
    }
}
```

Output:

```
5000.0
```

Step-by-step explanation

- `private double balance;` — balance ko class ke bahar directly access nahi kar sakte.
- `deposit()` — valid amount deposit karne ka controlled way.
- `getBalance()` — balance read karne ka public method.
- `account.deposit(5000)` — deposit method call hota hai.
- `account.getBalance()` — current balance return karta hai.

Agar aap likhen:

```
account.balance = 100000;
```

toh compile-time error aayega, kyunki `balance` private hai.

Encapsulation ke benefits

1. Data ko unauthorized direct access se protect karna.
2. Invalid values ko validation ke through rokna.
3. Code ko maintain karna easy banana.
4. Internal implementation ko change karna easier banana.

Interview answer: Encapsulation is the wrapping of data and methods into a single unit, with controlled access to data using access modifiers and methods such as getters and setters.

Remember: Encapsulation ka matlab sirf getters/setters banana nahi hai. Main goal data par controlled access dena hai.

4. Inheritance

Inheritance kya hai?

Definition: Inheritance is an OOP feature in which one class acquires accessible fields and methods from another class.

Hinglish mein: Jab ek class doosri class ki properties aur methods ko reuse karti hai, usse inheritance kehte hain.

- Parent class = Superclass
- Child class = Subclass
- `extends` keyword = Class inheritance ke liye use hota hai.

Real-life example: Ek `Vehicle` class mein common features hain, jaise start aur stop. `Car` aur `Bike` classes in features ko reuse kar sakti hain.

Java example

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}
```

Output:

```
Animal is eating
```

```
Dog is barking
```

Dry run

Animal — Parent class

`eat()`

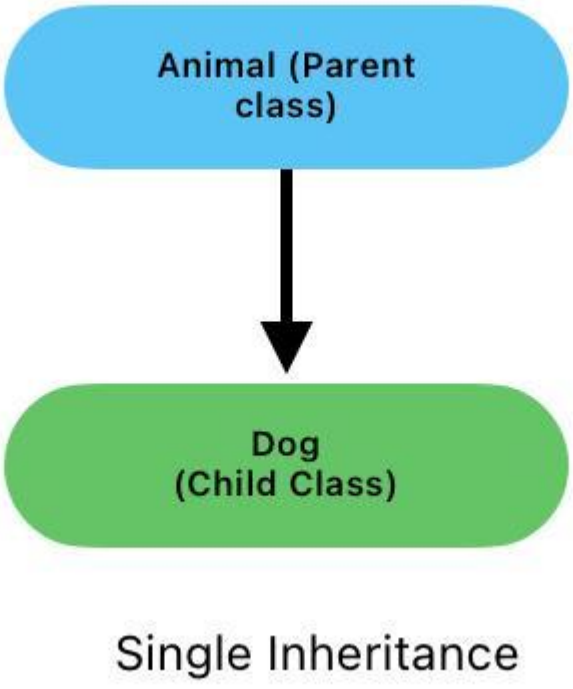
`extends`

Dog — Child class

`bark()` + inherited `eat()`

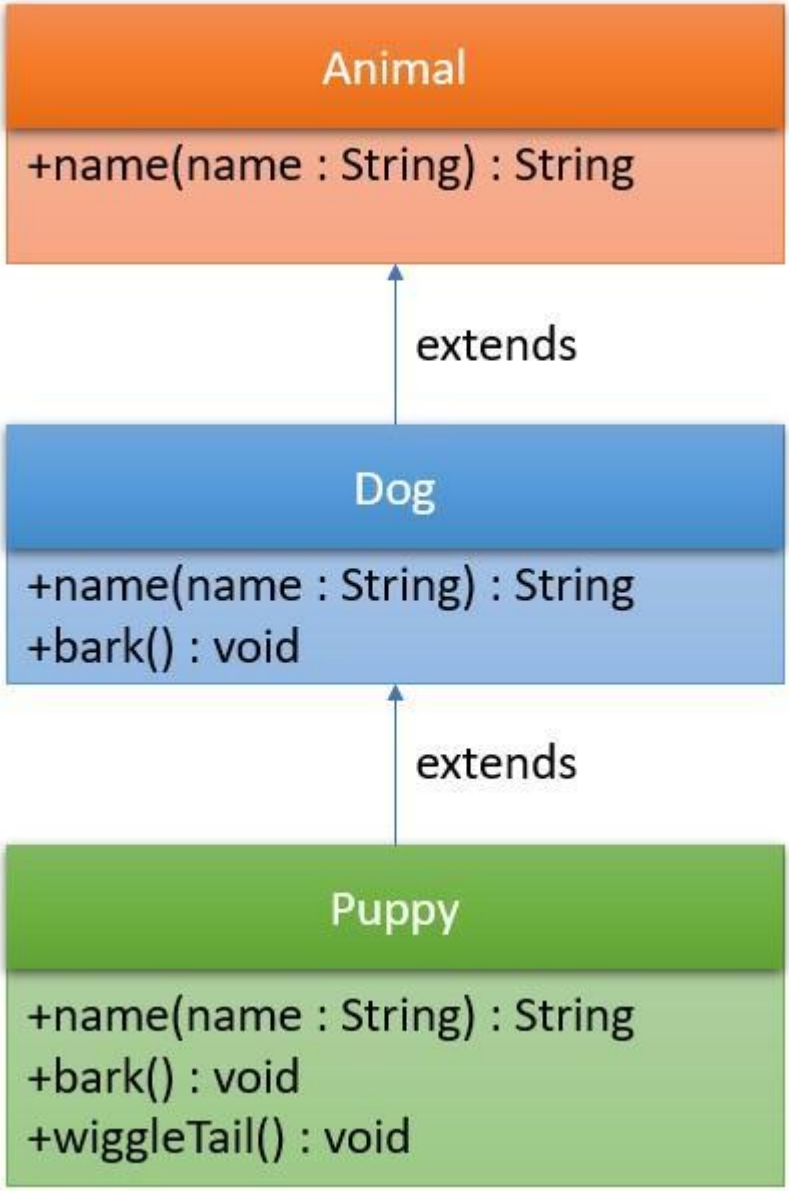
1. `Dog extends Animal` — Dog, Animal ki subclass hai.
2. `new Dog()` — Dog object create hota hai.
3. `d.eat()` — inherited public method call hota hai.
4. `d.bark()` — Dog ka apna method call hota hai.

Types of inheritance in Java



1. Single inheritance

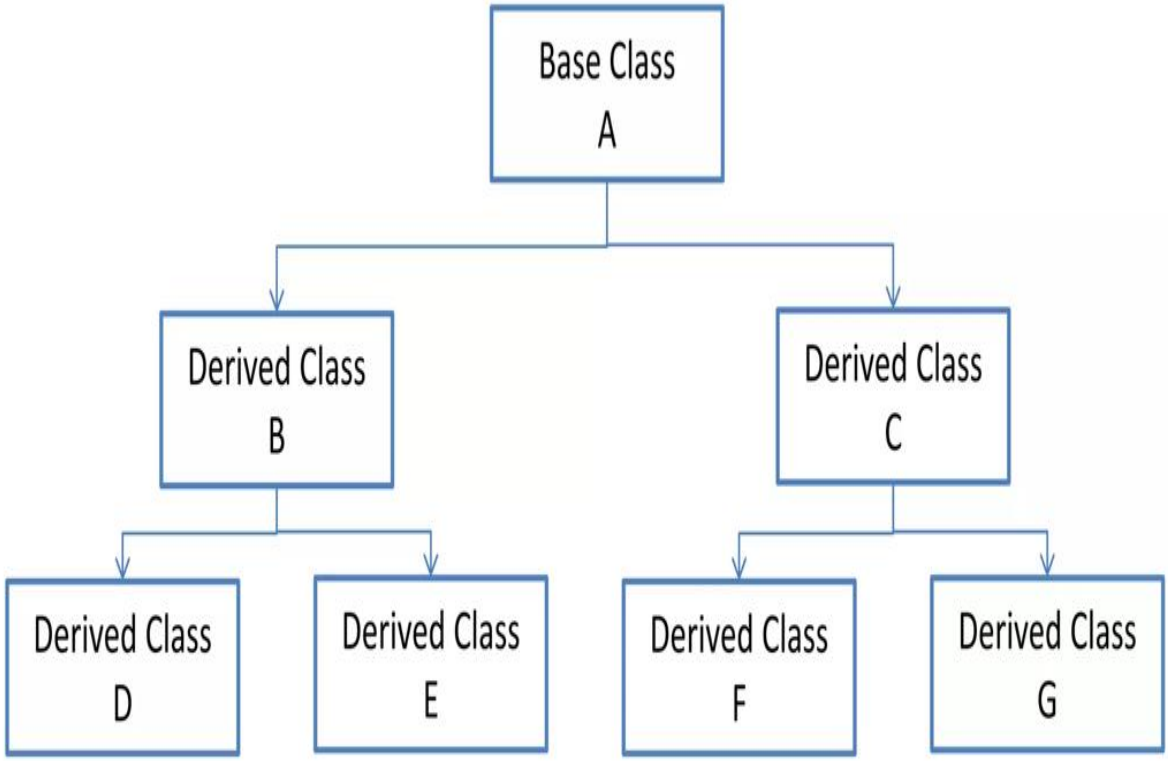
Ek child class ek parent class ko extend karti hai. Example: `Dog extends Animal`.



2. Multilevel inheritance

Ek chain hoti hai: `Animal → Dog → Puppy`.

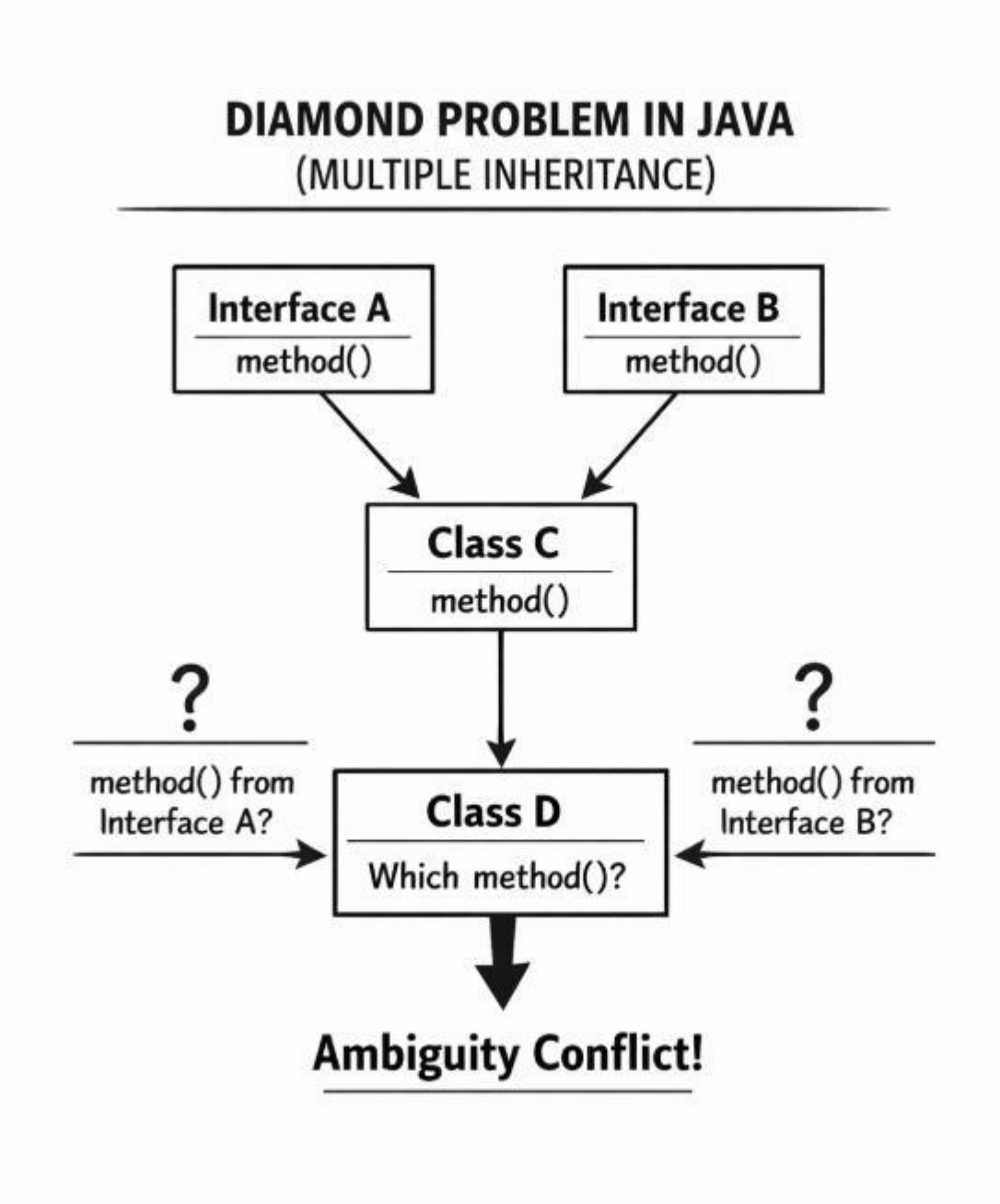
Hierarchical Inheritance



- In this type of inheritance, multiple derived classes inherits from a single base class.

3. Hierarchical inheritance

Ek parent class ki multiple child classes hoti hain, jaise `Animal → Dog` aur `Animal → Cat`.



4. Multiple inheritance

Ek class multiple parents se inherit kare. Java classes ke through ise directly support nahi karta, lekin multiple interfaces implement kar sakta hai.

Note: Hybrid inheritance, inheritance ke multiple patterns ka combination hai. Java mein class inheritance ke through ise directly implement nahi kar sakte agar iske liye multiple parent classes chahiye, lekin interfaces ka combination use kar sakte hain.

Inheritance ke benefits

- Code reusability
- Duplicate code kam hota hai
- Method overriding se specialization possible hoti hai
- Parent-child relationship represent hota hai

Interview answer: Inheritance allows a subclass to reuse accessible members of a superclass. In Java, class inheritance is implemented using `extends`, while interfaces are implemented using `implements`.

5. Polymorphism

Polymorphism kya hai?

Definition: Polymorphism means one name or interface can represent different behaviours depending on the context.

Hinglish mein: Ek hi method name ya interface alag situations mein alag behaviour de sakta hai.

Real-life example: Ek person ghar par son, office mein employee aur college reunion mein friend ho sakta hai. Role context ke hisaab se change hota hai.

Java mein polymorphism ke do important types hain:

A. Compile-time polymorphism (Method Overloading)

Same class mein same method name, lekin parameters different.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
```

```
        System.out.println(c.add(10, 20, 30));
    }
}
```

Output:

```
30
60
```

Compiler arguments aur parameter list ke basis par decide karta hai ki kaunsa `add()` method call hoga.

B. Runtime polymorphism (Method Overriding)

Child class parent class ke inherited instance method ko apni implementation se replace karti hai.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a;

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}
```

Output:

```
Dog barks
Cat meows
```

Runtime mein kya hota hai?

- `Animal a` — parent type ka reference variable.
- `a = new Dog()` — reference Dog object ko refer karta hai.
- `a.sound()` — Dog ka overridden method execute hota hai.
- `a = new Cat()` — ab reference Cat object ko refer karta hai.
- `a.sound()` — Cat ka overridden method execute hota hai.

Is behaviour ko dynamic method dispatch kehte hain.

Overloading vs Overriding

Feature	Overloading	Overriding
Meaning	Same method name, different parameter lists	Inherited instance method ki new implementation
Usually occurs	Same class mein	Parent-child classes mein
Parameters	Different	Same signature
Polymorphism	Compile-time	Runtime
Return type	Parameters/signature ke rules follow hone chahiye	Same ya covariant return type
Inheritance required?	No	Yes, normal class overriding ke liye

Important: Sirf return type change karke method overload nahi hota. `static` methods class ke level par hide hote hain; woh instance-method overriding nahi hai.

Interview answer: Polymorphism allows the same method name or reference type to exhibit different behaviours. Java supports compile-time polymorphism through overloading and runtime polymorphism through method overriding.

6. Abstraction

Abstraction kya hai?

Definition: Abstraction means exposing essential functionality while hiding unnecessary implementation details.

Hinglish mein: User ko sirf zaroori functionality dikhana aur internal implementation ko hide karna abstraction hai.

Real-life example: Aap ATM se paise withdraw karte ho. Aapko internal banking server, transaction validation aur database processing ka poora implementation nahi dekhna padta.

Java mein abstraction achieve karne ke do main ways hain:

1. Abstract class
2. Interface

A. Abstract class

`abstract` keyword se abstract class banate hain. Ismein abstract methods (without body) aur concrete methods (with body) dono ho sakte hain.

```
abstract class Vehicle {
    abstract void start();

    void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with a key");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();

        v.start();
        v.stop();
    }
}
```

Output:

```
Car starts with a key
Vehicle stopped
```

Explanation:

- `abstract class Vehicle` — abstract class declare ki.
- `abstract void start();` — method declaration hai, body nahi hai.
- `Car extends Vehicle` — child class abstract class ko extend karti hai.
- `Car start()` ko implement karti hai.
- Abstract class ka direct object nahi bana sakte: `new Vehicle()` invalid hai.

B. Interface

Interface ek contract define karta hai jise implementing class follow karti hai.

```
interface Payment {
    void pay(double amount);
```

```
}

class UpiPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using UPI: " + amount);
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p = new UpiPayment();
        p.pay(500);
    }
}
```

Output:

```
Paid using UPI: 500.0
```

Interface mein declared ordinary abstract methods ko implementing class generally `public` implementation deti hai.

Abstract class vs Interface

Abstract class	Interface
`extends` keyword	`implements` keyword
Abstract aur concrete methods dono	Abstract methods ke saath default, static aur private methods bhi ho sakte hain
Instance fields ho sakte hain	Fields implicitly `public static final` hote hain
Class sirf ek class extend kar sakti hai	Class multiple interfaces implement kar sakti hai
Related classes ke common state aur behaviour ke liye useful	Common contract define karne ke liye useful

Interview answer: Abstraction hides implementation details and exposes essential functionality. In Java, abstraction is achieved using abstract classes and interfaces.

Encapsulation vs Abstraction

- Encapsulation: Data aur methods ko ek unit mein organize karke access control karna.
- Abstraction: Unnecessary implementation details hide karke essential operations expose karna.

Dono related hain, lekin same concept nahi hain.

7. Access Modifiers

Access modifiers kya hote hain?

Definition: Access modifiers control the visibility and accessibility of classes, methods, constructors and fields.

Hinglish mein: Access modifiers decide karte hain ki kisi variable, method ya constructor ko kahan se access kar sakte hain.

Java mein four main access levels hain:

- `public`
- `protected`
- default (koi modifier nahi)
- `private`

Comparison table

Modifier	Same class	Same package	Subclass in another package	Other package, non-subclass
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes, through inheritance rules	No
default	Yes	Yes	No	No
`private`	Yes	No	No	No

Note: Cross-package `protected` access subclass ke context mein allowed hota hai, lekin har situation mein arbitrary parent-class object ke through access allowed nahi hota.

A. Public

Kahin se bhi accessible, subject to normal Java and module access rules.

```
public class Student {
    public String name = "Suhana";
}
```

B. Private

Sirf declaring class ke andar accessible.

```
class Student {
    private int age = 21;

    public int getAge() {
        return age;
    }
```

```
}
```

Yahan `age` ko direct class ke bahar access nahi kar sakte. `getAge()` ke through read kar sakte hain.

C. Protected

Same package mein accessible; different package mein subclass ko inheritance-related conditions ke saath accessible.

```
class Animal {
    protected void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void display() {
        eat();
    }
}
```

`Dog` inherited protected method `eat()` ko use kar sakti hai.

D. Default (package-private)

Agar access modifier nahi likhte, member same package tak accessible hota hai.

```
class Student {
    void study() {
        System.out.println("Student is studying");
    }
}
```

Yahan `study()` default/package-private access rakhta hai.

8. Methods and Method Overloading

Method kya hota hai?

Definition: A method is a block of code that performs a specific task and can be called when needed.

Hinglish mein: Method code ka reusable block hai jo ek particular task perform karta hai.

Basic method syntax

```
returnType methodName(parameters) {
    // code to execute
```

```
    return value;
}
```

Example:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        int result = c.add(10, 20);
        System.out.println(result);
    }
}
```

Output:

```
30
```

Method ke important parts

- `int` — return type; method integer result return karega.
- `add` — method name.
- `int a, int b` — parameters.
- `return a + b` — calculated result caller ko wapas deta hai.
- `c.add(10, 20)` — method call hai.

Types of methods based on return and parameters

1. No parameters, no return value

```
void greet() {
    System.out.println("Hello");
}
```

2. Parameters, no return value

```
void greet(String name) {
    System.out.println("Hello " + name);
}
```

3. No parameters, returns a value

```
int getNumber() {
    return 10;
}
```

4. Parameters and returns a value

```
int square(int n) {
    return n * n;
}
```

Method overloading kya hai?

Definition: Method overloading means defining multiple methods with the same name but different parameter lists in the same class or inherited method context.

Hinglish mein: Same naam ke multiple methods banana, jinke parameters ki quantity ya types ya order different ho, method overloading hai.

Example: Three overloaded methods

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
        System.out.println(c.add(2.5, 3.5));
    }
}
```

Output:

```
30
60
6.0
```

Compiler kaise decide karta hai?

Method call	Selected method	Reason
`add(10, 20)`	`add(int, int)`	Two integer arguments

`add(10, 20, 30)`	`add(int, int, int)`	Three integer arguments
`add(2.5, 3.5)`	`add(double, double)`	Two double arguments

Compiler compile time par arguments aur available method signatures ke basis par suitable method resolve karta hai.

Valid overloading examples

```
void display(int a) { }

void display(String a) { }
```

Parameters ke types different hain.

```
void display(int a, int b) { }

void display(int a) { }
```

Parameters ki quantity different hai.

```
void display(int a, String b) { }

void display(String a, int b) { }
```

Parameters ka order different hai.

Invalid overloading example

```
int calculate(int a) {
    return a;
}

// Compile-time error: same signature
double calculate(int a) {
    return a * 2.0;
}
```

Sirf return type change karne se method overload nahi hota.

Methods vs constructors

Method	Constructor
Koi valid method name ho sakta hai	Class ke naam jaisa hota hai
Return type hota hai, including `void`	Koi return type nahi hota
Task perform karta hai	Object initialize karta hai
Explicitly call kiya jata hai	Object creation ke time invoke hota hai
Inherited/overridden ho sakta hai, subject to rules	Inherit ya override nahi hota

Interview answer: Method overloading is compile-time polymorphism in which multiple methods share the same name but have different parameter lists. Return type alone cannot distinguish overloaded methods.

9. Quick Revision — Interview ke liye

OOP concept	Main purpose	Java keyword / feature
Class and Object	Structure aur instances create karna	`class`, `new`
Constructor	Object initialize karna	Class-name constructor
Encapsulation	Data access control	`private`, getters/setters
Inheritance	Code reuse	`extends`
Polymorphism	Different behaviours	Overloading, overriding
Abstraction	Implementation details hide karna	`abstract`, `interface`
Access modifiers	Visibility control karna	`public`, `protected`, default, `private`
Methods	Reusable tasks perform karna	Return type, method name, parameters
Method overloading	Same name, different parameter lists	Compile-time polymorphism

OOP ke four pillars

Interview mein agar poochhein, “What are the four pillars of OOP?”, toh answer hai:

1. Encapsulation — data ko protect aur access control karna.
2. Inheritance — existing code ko reuse karna.
3. Polymorphism — ek interface ya method ke different behaviours.
4. Abstraction — essential functionality show karna, implementation hide karna.

Tasks

 Create Student and Employee classes.

 Add properties and methods.

 Demonstrate constructors, inheritance, and encapsulation.

• Create Student and Employee classes.

```
class Student {
    // Properties
    String name;
    int rollNo;
    String course;

    // Method
    void displayStudent() {
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Course: " + course);
    }
}

class Employee {
    // Properties
    String name;
    int employeeId;
    double salary;

    // Method
    void displayEmployee() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Salary: " + salary);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        // Student object
        Student s1 = new Student();
        s1.name = "Suhana";
        s1.rollNo = 101;
        s1.course = "Computer Engineering";

        // Employee object
        Employee e1 = new Employee();
        e1.name = "Rahul";
        e1.employeeId = 501;
        e1.salary = 35000;

        // Display details
        s1.displayStudent();

        System.out.println();

        e1.displayEmployee();
    }
}
```

2] Add properties and methods.

```
class Student {
    // Properties
    String name;
```

```
    int rollNo;
    String course;
    double marks;

    // Method to display student details
    void displayDetails() {
        System.out.println("Student Name: " + name);
        System.out.println("Roll Number: " + rollNo);
        System.out.println("Course: " + course);
        System.out.println("Marks: " + marks);
    }

    // Method to check result
    void checkResult() {
        if (marks >= 35) {
            System.out.println("Result: Pass");
        } else {
            System.out.println("Result: Fail");
        }
    }
}

class Employee {
    // Properties
    String name;
    int employeeId;
    String department;
    double salary;
```

```
    // Method to display employee details
    void displayDetails() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
    }

    // Method to calculate annual salary
    void calculateAnnualSalary() {
        double annualSalary = salary * 12;
        System.out.println("Annual Salary: " + annualSalary);
    }
}

public class Main {
    public static void main(String[] args) {

        // Create Student object
        Student s1 = new Student();
        s1.name = "Suhana";
        s1.rollNo = 101;
        s1.course = "Computer Engineering";
        s1.marks = 85;

        System.out.println("--- Student Details ---");
        s1.displayDetails();
        s1.checkResult();
```

```
        // Create Employee object
        Employee e1 = new Employee();
        e1.name = "Rahul";
        e1.employeeId = 501;
        e1.department = "IT";
        e1.salary = 35000;

        System.out.println("\n--- Employee Details ---");
        e1.displayDetails();
        e1.calculateAnnualSalary();
    }
}
```

3] Demonstrate constructors, inheritance, and encapsulation.

```
class Person {
    // Encapsulation: private data
    private String name;
    private int age;

    // Parameterized constructor
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter methods
    public String getName() {
        return name;
```

```
    }

    public int getAge() {
        return age;
    }
}

// Inheritance: Student inherits Person
class Student extends Person {
    private int rollNo;

    // Student constructor
    Student(String name, int age, int rollNo) {
        super(name, age);
        this.rollNo = rollNo;
    }

    public void display() {
        System.out.println("Name: " + getName());
        System.out.println("Age: " + getAge());
        System.out.println("Roll No: " + rollNo);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Suhana", 21, 101);

        s1.display();
```